

多人脸采集素材自动化清洗系统 — 概要设计文档

1. 设计概述

1.1 设计目标

本文档从系统架构、功能模块划分、数据流设计和接口规范四个层面对"多人脸采集素材自动化清洗系统"进行概要设计，为后续详细设计和编码实现提供技术蓝图。设计遵循以下原则：**模块化**——每个功能模块职责单一、边界清晰；**可降级**——核心引擎均设计有主方案和降级方案，确保系统在依赖缺失时仍可运行；**前后端分离**——前端为纯静态页面，后端通过 RESTful API 提供服务。

1.2 系统定位

系统是一套面向车载座舱多人采集场景的离线数据清洗工具，以本地 Web 服务形式部署，通过浏览器访问。核心价值在于将人脸检测、特征匹配、座位分类、可视化标注整合为端到端自动化流水线。配套 Android 原生客户端通过 WebView 封装，支持手机扫码快速连接服务器。

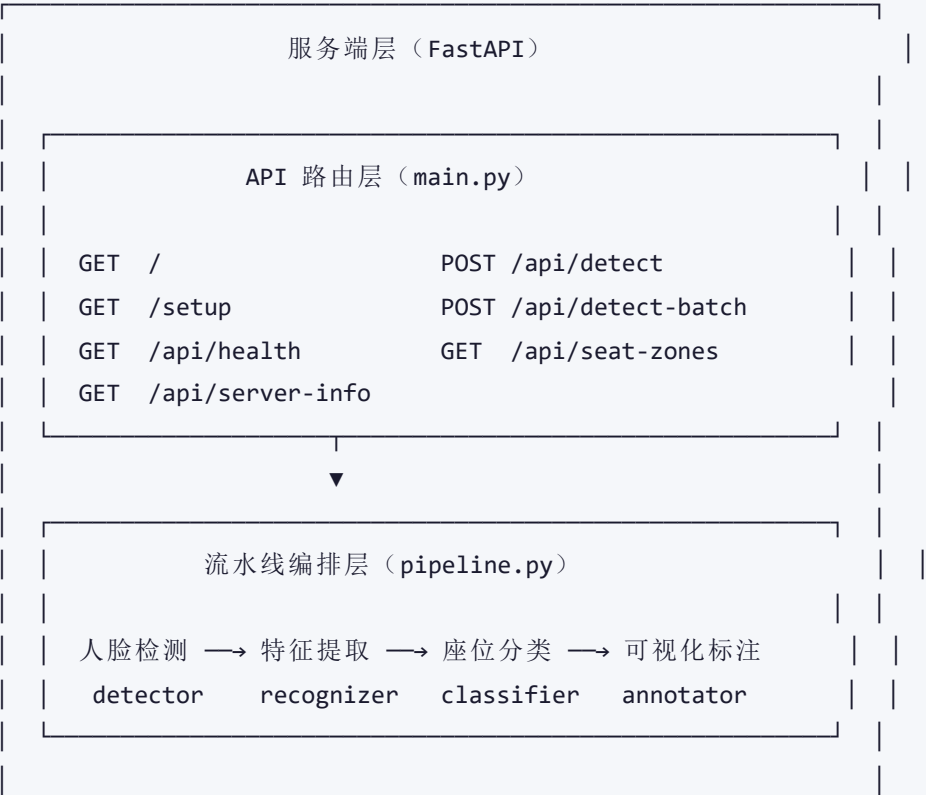
2. 系统架构

2.1 总体架构

系统采用 B/S (Browser/Server) 架构，分为客户端层和服务端层两个层次。



HTTP



detector.py	recognizer.py	seat_classifier.py
YOLOv8-face	InsightFace	SeatClassifier
Haar fallback	像素 fallback	FaceAnnotator
技术栈：FastAPI + Uvicorn + YOLOv8-face + InsightFace + PIL		

2.2 技术选型说明

后端框架选择 FastAPI，原因在于其原生支持异步端点、自动 OpenAPI 文档生成、基于 Pydantic 的请求验证，且性能在 Python Web 框架中处于第一梯队。配合 Uvicorn ASGI 服务器，可高效处理文件上传等 IO 密集型任务。

人脸检测引擎以 YOLOv8n-face (lindevs 版本，约 6MB) 为主方案，基于 ultralytics 库加载。当模型文件不存在或 ultralytics 库未安装时，自动降级至 OpenCV Haar Cascade——后者精度较低但无需额外模型下载，适合快速验证和环境受限场景。检测置信度阈值默认 0.1（可通过参数调整）。

人脸特征提取设计了两级方案。主方案为 InsightFace ArcFace (buffalo_l 模型) 512 维嵌入向量。降级方案为基于 OpenCV 的像素特征——将人脸区域裁剪为 16x16 灰度图，进行直方图均衡后展平为 256 维向量并 L2 归一化。**注：**特征匹配默认关闭（`use_recognition=False`），批量场景下如需跨图匹配同一人，需在初始化 Pipeline 时显式启用。

前端技术选择原生 HTML5 + CSS3 + JavaScript 单文件方案，不依赖任何构建工具或框架。包含 `index.html`（检测主页面）和 `setup.html`（配网页面）两个文件。响应式设计适配移动端，支持拖拽上传。

标注引擎使用 PIL (Pillow) 而非 OpenCV 的 `putText`，原因是 OpenCV 不支持直接渲染中文字符。中文标签依赖系统字体（Windows 优先 `simhei.ttf` / `msyh.ttc`，Linux 优先 `wqy-zenhei`）。

3. 功能模块设计

3.1 模块总览

系统由五个后端模块和两个前端页面组成：



3.2 模块一：API 路由层 (main.py)

职责：定义 HTTP 端点，处理请求解析和响应封装，管理静态文件服务。

包含端点：

- `GET /` : 返回前端单页应用 (`frontend/index.html`) , 作为静态文件服务。
- `GET /setup` : 返回配网页面 (`frontend/setup.html`) 。
- `GET /api/health` : 健康检查接口, 返回 `{"status": "ok", "message": "多人脸检测服务运行中"}` , 用于监控系统探活。
- `POST /api/detect` : 单张图像检测接口。接收 `multipart/form-data` 格式的图片文件 (字段名 `file`) , 调用 `Pipeline.process_image` 处理后返回 JSON 结果。对无效输入返回 HTTP 400, 对内部错误返回 HTTP 500。
- `POST /api/detect-batch` : 批量检测接口。接收多个图片文件 (字段名 `files`) , 调用 `Pipeline.process_batch` 处理, 返回所有图片的检测结果。
- `GET /api/seat-zones` : 座位区域配置查询接口, 返回当前生效的五个座位区域的坐标范围、中文标签和对应颜色。
- `GET /api/server-info` : 服务器地址查询接口, 返回可供客户端连接的服务器 URL (优先公网 IP, 降级为局域网 IP) , 供配网页面和 Android 客户端使用。

设计要点: 路由层不包含任何业务逻辑, 只负责请求/响应转换和错误处理。文件类型校验在路由层完成, 仅接受可被 `cv2.imdecode` 解析的图片格式。路由层还挂载了 `frontend/` 目录为静态文件服务 (`/static` 路径)。

数据模型 (Pydantic) :

```
class FaceResult(BaseModel):
    bbox: list[float]
    conf: float
    seat: str
    seat_label: str
    person_id: int

class DetectionResponse(BaseModel):
    image_id: int
    filename: str
    num_faces: int
    faces: list[FaceResult]
    annotated_image_base64: str
    processing_time_ms: float
```

3.3 模块二: 流水线编排引擎 (pipeline.py)

职责: 协调人脸检测、特征提取、座位分类、可视化标注四个引擎按正确顺序执行。

核心类 Pipeline:

```
class Pipeline:
    def __init__(self, detector_conf=0.5, similarity_threshold=0.5,
use_recognition=False):
        self.detector = FaceDetector(conf_threshold=detector_conf)
        self.recognizer = FaceRecognizer() if use_recognition else None
        self.classifier = SeatClassifier()
        self.annotator = FaceAnnotator()
```

`process_image(image, image_id=0)` 方法:

1. 调用 `FaceDetector.detect()` 检测所有人脸
2. 若 recognizer 启用, 提取每个人脸的特征向量 (embedding)
3. 若 recognizer 启用, 调用 `match_faces_across_images()` 分配 person_id; 否则按检测顺序分配 1, 2, 3...
4. 调用 `SeatClassifier.classify()` 对每个人脸进行座位分类
5. 调用 `FaceAnnotator.draw()` 绘制可视化标注
6. 返回结构化结果字典 (含 annotated_image numpy 数组)

`process_batch(images)` 方法:

1. 对每张图片执行检测 (+可选的 embedding 提取)
2. 若 recognizer 启用, 对所有图片的人脸执行全局跨图匹配
3. 按 image_id 重新分组, 对每组执行座位分类和标注
4. 返回 `(results_list, annotated_images)` 二元组, 其中 results_list 为每张图片的结构化结果, annotated_images 为对应的标注后 OpenCV 图像数组

设计要点: Pipeline 持有四个引擎的实例引用。 `use_recognition` 默认为 False, 在不需要跨图身份匹配的场景下可跳过 InsightFace 加载, 显著降低 CPU 占用和启动时间。

3.4 模块三: 人脸检测引擎 (detector.py)

职责: 在输入图像中定位所有人脸, 返回边界框和置信度。

核心类 FaceDetector:

```
class FaceDetector:
    def __init__(self, model_path=None, conf_threshold=0.5):
```

初始化时按以下优先级尝试加载 YOLOv8-face 模型:

1. 用户指定的 `model_path`
2. `yolov8n-face-lindevs.pt` (当前项目推荐模型, 约 6MB)

3. yolov8n-face.pt

若以上模型文件均不存在，或 ultralytics 库未安装，则加载 OpenCV 内置的 Haar Cascade 分类器（haarcascade_frontalface_default.xml）。

detect(image) 方法：

- YOLO 引擎：调用 `model.predict()` 推理，过滤低于 `conf_threshold` 的检测结果，输出统一格式
- Haar 引擎：图像转灰度后执行多尺度检测，使用检测面积与图像面积之比映射为 0.5-0.99 的伪置信度

两种引擎的输出格式统一为 `{"bbox": [x1, y1, x2, y2], "conf": float}`，并按 bbox 的 x1 坐标从左至右排序。

属性： `is_yolo` — 返回当前是否使用 YOLOv8 后端。

3.5 模块四：特征提取与身份匹配引擎 (recognizer.py)

职责： 从裁剪后的人脸图像中提取特征向量，并通过余弦相似度实现跨图片的身份匹配。

核心类 FaceRecognizer：

```
class FaceRecognizer:
    def __init__(self):
        self.app = None
        self.available = False
        self._load_model()
```

_load_model() 方法： 尝试加载 InsightFace 的 FaceAnalysis (buffalo_l 模型，CPU 模式)。加载失败时设置 `available = False`，打印降级警告。

extract_embedding(image, bbox) 方法：

- 主方案 (InsightFace)：将 bbox 扩展 20% 后裁剪人脸区域，送入 FaceAnalysis 获取 512 维 `normed_embedding`
- 降级方案：裁剪 ROI → 缩放至 16x16 → 灰度 → 直方图均衡 → 展平为 256 维向量 → L2 归一化

match_faces_across_images(all_faces, similarity_threshold=0.5) 方法： 维护一个 `person_db` 字典（`person_id` → 特征向量列表）。遍历所有人脸，计算当前人脸特征与 `person_db` 中每个已知身份的余弦相似度。若最高相似度超过阈值，则分配对应 `person_id` 并将新特征向量加入该身份的特征列表；否则创建新身份，`person_id` 递增。

设计要点： `person_db` 的生命周期限定在单次 `match_faces_across_images` 调用内。无 embedding 的人脸（提取失败）直接分配新 `person_id`。 `cosine_similarity()` 为静态方法，支持 None 输入安全处理。

3.6 模块五：座位分类与标注引擎 (seat_classifier.py)

职责： 根据人脸边界框坐标和大小进行座位区域分类，以及在图像上绘制可视化标注。

座位区域定义 (SEAT_ZONES)：

座位ID	X范围	Y范围	标签	颜色	----- ----- ----- ----- -----	co_driver	0.00-0.40
0.30-0.55	副驾	橙色	master_driver	0.60-1.00	0.30-0.55	主驾	青色
posterior_left	0.25-0.45	0.25-0.45	后排左	红色	posterior_right	0.45-0.65	0.25-0.45
后排中	绿色	after_the_middle	0.65-0.90	0.25-0.50	后排右	蓝色	

SeatClassifier 类：

`classify(bbox, img_w, img_h, all_bboxes)` 方法采用自适应前后排判断策略：

1. 仅有一张人脸时：根据人脸宽度与图像宽度之比判断 (> 12% → 前排)
2. 两张人脸时：若 Y 坐标跨度 > 12% → 按 Y 坐标高低区分前后排；否则按平均宽度判断
3. 三张人脸及以上：使用 K-means (K=2) 对 Y 坐标聚类，结合人数分布和宽度信息判定前后排
4. 确定前后排后，按 X 中心坐标映射到对应座位区域

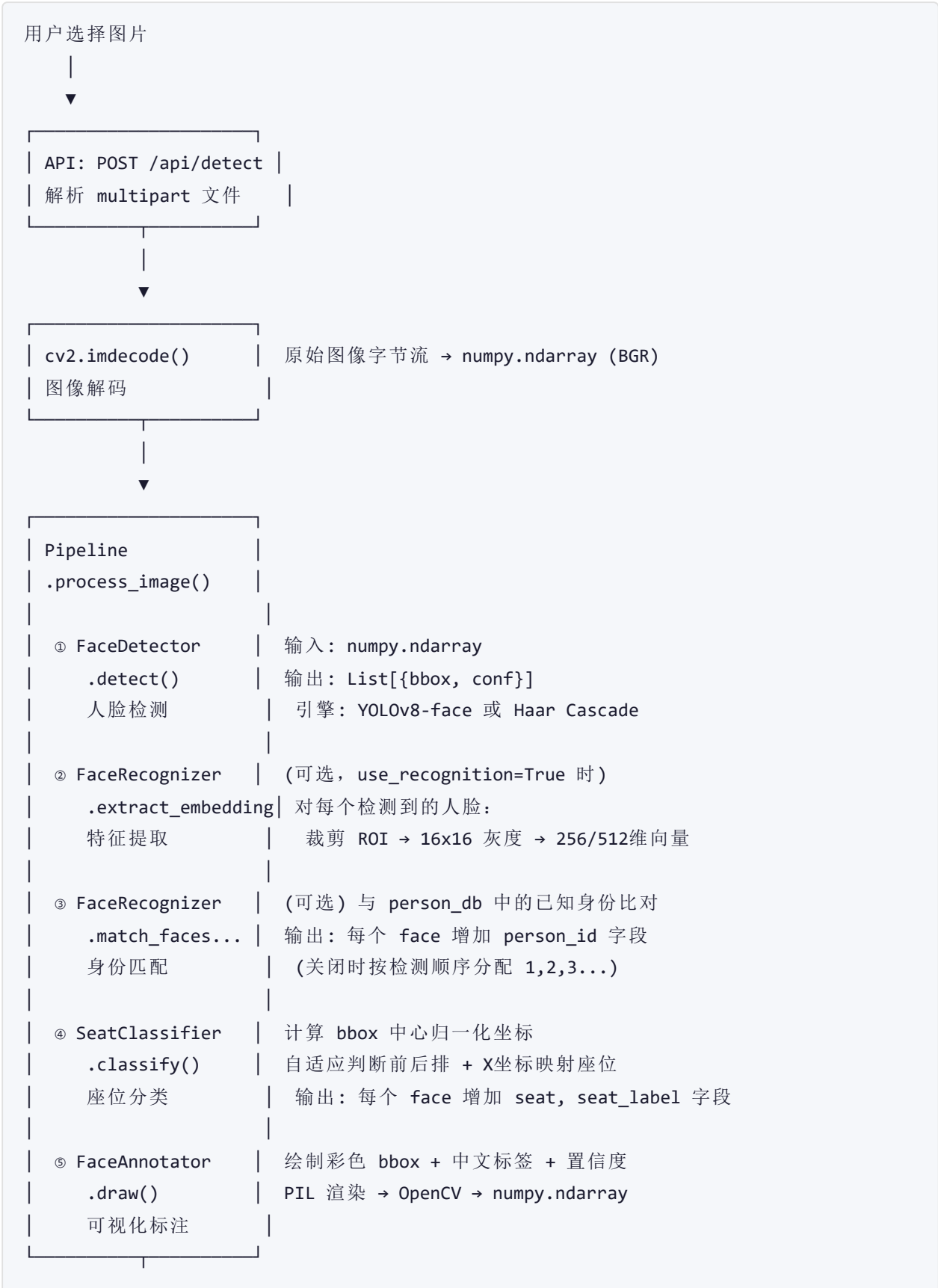
FaceAnnotator 类：

`draw(image, faces)` 方法将 OpenCV 图像 (BGR) 转换为 PIL 图像 (RGB)，使用 ImageDraw 为每个人脸绘制：彩色矩形框、座位名称+`person_id` 标签 (带彩色背景)、置信度数值。中文渲染依赖系统字体，字体查找顺序：`simhei.ttf` → `msyh.ttc` → `simsun.ttc` (Windows) → `wqy-zenhei` (Linux) → `DejaVuSans` (fallback)。标注完成后转回 OpenCV BGR 格式。

`draw_seat_zones(image, alpha=0.15)` 方法绘制半透明的座位区域热力图，用于调试和可视化区域覆盖范围。

4. 数据流设计

4.1 单张检测数据流





路由层组装响应	
返回 JSON	

annotated_image → JPEG → Base64

{image_id, filename, num_faces,

faces[], annotated_image_base64,

processing_time_ms}

4.2 批量处理数据流

用户上传 N 张图片



```
API: POST /api/detect-batch |
解析多个 multipart 文件 |
```



```
Pipeline
.process_batch(images)

阶段一：逐图检测
for each image:
    detector.detect()
    (可选) recognizer
        .extract_embedding
收集全部人脸到 all_faces
(标记 image_id)
```



```
阶段二：全局身份匹配
(use_recognition=True)
match_faces_across_images
遍历 all_faces, 统一分配
person_id
```



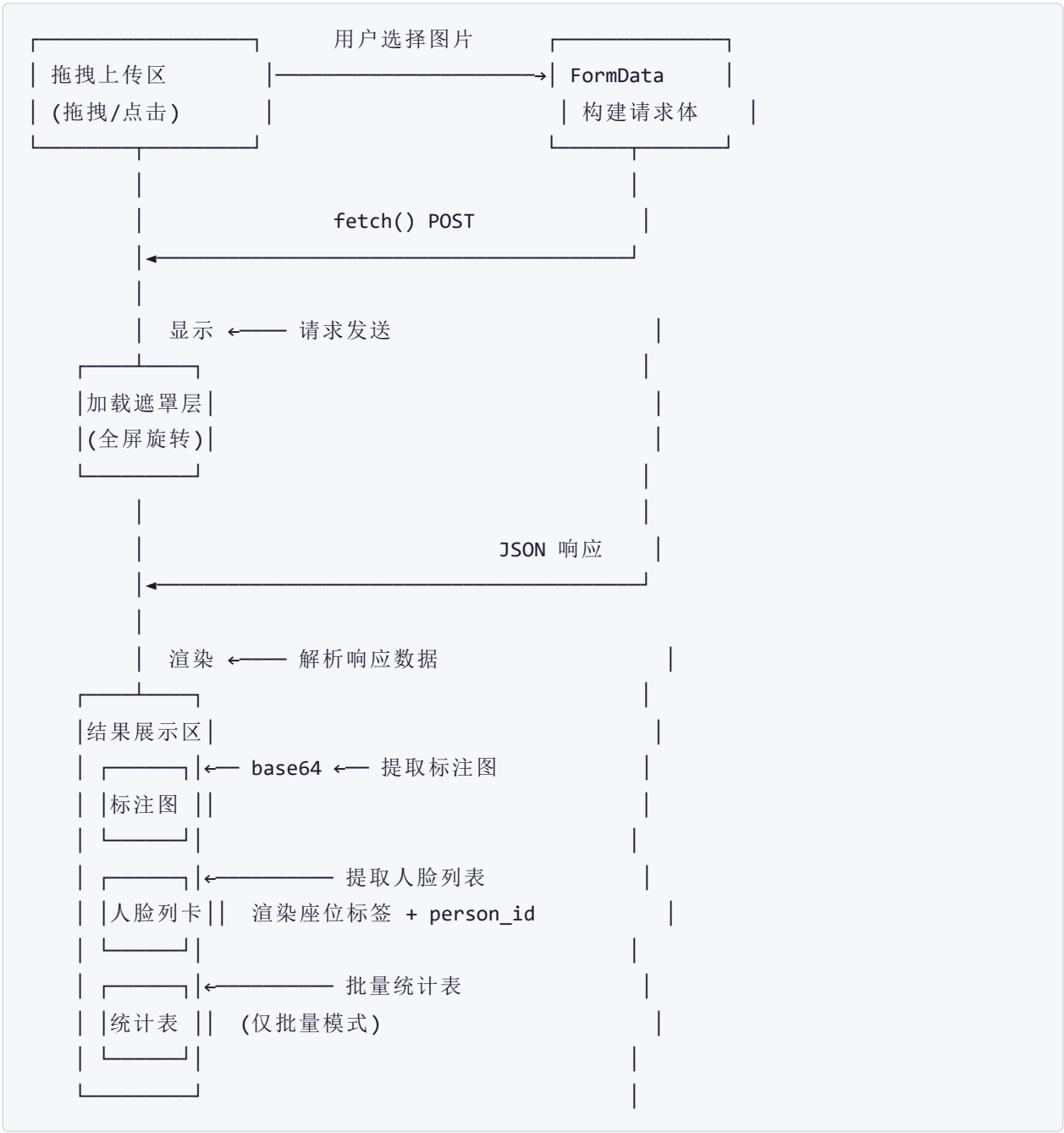
```
阶段三：逐图分类+标注
for each image:
    按 image_id 筛选人脸
    classifier.classify()
    annotator.draw()
    生成 per_image_result
```



```
组装 JSON 响应
{total_images, total_faces|
processing_time_ms,
results[]}
```

(注：当前不返回聚合统计)

4.3 前端交互数据流



5. 接口设计

5.1 API 接口规范

所有 API 遵循 RESTful 风格。文件上传使用 `multipart/form-data`，其余请求和响应均使用 JSON 格式。错误响应统一格式为 `{"detail": "错误描述信息"}`。

GET /api/health

响应 200:

```
{
  "status": "ok",
  "message": "多人脸检测服务运行中"
}
```

GET /api/seat-zones

响应 200:

```
{
  "zones": {
    "co_driver": {"x_range": [0.00, 0.40], "y_range": [0.30, 0.55], "label": "副驾"},
    "master_driver": {"x_range": [0.60, 1.00], "y_range": [0.30, 0.55], "label": "主驾"},
    "posterior_left": {"x_range": [0.25, 0.45], "y_range": [0.25, 0.45], "label": "后排左"},
    "posterior_right": {"x_range": [0.45, 0.65], "y_range": [0.25, 0.45], "label": "后排中"},
    "after_the_middle": {"x_range": [0.65, 0.90], "y_range": [0.25, 0.50], "label": "后排右"}
  },
  "colors": {
    "posterior_left": [255, 100, 100],
    "posterior_right": [100, 255, 100],
    "after_the_middle": [100, 100, 255],
    "co_driver": [255, 200, 100],
    "master_driver": [100, 200, 255]
  }
}
```

GET /api/server-info

响应 200:

```
{
  "url": "http://192.168.1.100:8000"
}
```

优先返回 `PUBLIC_IP` 环境变量配置的公网地址；未配置时返回局域网 IP。

POST /api/detect

请求: `multipart/form-data` , 字段 `file` = 图片文件

响应 200:

```
{
  "image_id": 0,
  "filename": "car_01.jpg",
  "num_faces": 3,
  "faces": [
    {
      "bbox": [120, 80, 200, 170],
      "conf": 0.92,
      "seat": "master_driver",
      "seat_label": "主驾",
      "person_id": 1
    }
  ],
  "annotated_image_base64": "data:image/jpeg;base64,/9j/4AAQ...",
  "processing_time_ms": 2340.5
}
```

错误 400: `{"detail": "无法解析图片文件，请上传JPG/PNG格式"}` 错误 500: `{"detail": "处理失败：..."}`

POST /api/detect-batch

请求: `multipart/form-data` , 字段 `files` = 多个图片文件

响应 200:

```
{
  "total_images": 3,
  "total_faces": 8,
  "processing_time_ms": 8500.0,
  "results": [
    {
      "image_id": 0,
      "filename": "img1.jpg",
      "num_faces": 3,
      "faces": [
        {
          "bbox": [120, 80, 200, 170],
          "conf": 0.92,
          "seat": "master_driver",
          "seat_label": "主驾",
          "person_id": 1
        }
      ],
      "annotated_image_base64": "data:image/jpeg;base64,..."
    }
  ]
}
```

注：当前批量接口不返回聚合统计（如每人出现次数、主要座位）。如需此功能，可在前端对 results 数据自行汇总。

5.2 内部模块接口

```
# detector.py
class FaceDetector:
    def __init__(self, model_path=None, conf_threshold=0.5): ...
    def detect(self, image: np.ndarray) -> list[dict]:
        # 返回: [{"bbox": [x1,y1,x2,y2], "conf": float}, ...]
    @property
    def is_yolo(self) -> bool: ...

# recognizer.py
class FaceRecognizer:
    def __init__(self): ...
    @property
    def available(self) -> bool: ...
    def extract_embedding(self, image: np.ndarray, bbox: list[float]) ->
Optional[np.ndarray]:
        # 返回: 512维 (InsightFace) 或 256维 (fallback) 单位特征向量, 失败返回 None
    @staticmethod
    def cosine_similarity(emb1, emb2) -> float: ...
    def match_faces_across_images(self, all_faces: list[dict],
similarity_threshold=0.5) -> list[dict]:
        # 输入: 包含 embedding 和 image_id 字段的人脸列表
        # 输出: 增加 person_id 字段后返回

# seat_classifier.py
class SeatClassifier:
    @staticmethod
    def classify(bbox, img_w: int, img_h: int, all_bboxes: list = None) -> str:
        # 返回座位 ID, 如 "master_driver"
    @staticmethod
    def get_label(seat_id: str) -> str: ...
    @staticmethod
    def get_color(seat_id: str) -> tuple: ...

class FaceAnnotator:
    @staticmethod
    def draw(image: np.ndarray, faces: list[dict]) -> np.ndarray: ...
    @staticmethod
    def draw_seat_zones(image: np.ndarray, alpha=0.15) -> np.ndarray: ...

# pipeline.py
class Pipeline:
    def __init__(self, detector_conf=0.5, similarity_threshold=0.5,
```



```
use_recognition=False): ...
    def process_image(self, image: np.ndarray, image_id=0) -> dict:
        # 返回: {"image_id", "num_faces", "faces", "annotated_image"}
    def process_batch(self, images: list[np.ndarray]) -> tuple[list[dict],
list[np.ndarray]]:
        # 返回: (results_list, annotated_images)
```

6. 错误处理设计

6.1 错误分类

系统将可能出现的错误分为三类：

输入错误 (HTTP 400)：用户上传的文件不是有效图片（cv2.imdecode 返回 None）、上传文件数量为零。

处理错误 (HTTP 500)：检测引擎运行时异常、特征提取失败、标注绘制异常。这类错误被 try-except 捕获后返回友好的错误消息，不会导致服务进程终止。

系统错误：端口占用、依赖库缺失等启动阶段错误。通过降级机制和启动日志提示处理。

6.2 降级策略

人脸检测（启动时自动降级）：

尝试加载 YOLOv8-face 模型文件

|
├─ 成功 → 使用 YOLO 引擎（日志：[FaceDetector] YOLOv8-face 就绪）
|
└─ 失败（文件不存在或库未安装）

▼
加载 Haar Cascade → 使用 Haar 引擎（日志：[FaceDetector] Haar Cascade 就绪）

特征提取（按需降级）：

use_recognition=True 时加载 InsightFace

|

├─ 成功 → 使用 ArcFace 512维特征

|

(日志: [FaceRecognizer] InsightFace ArcFace 就绪)

|

└─ 失败 (库未安装或模型未下载)

|

▼

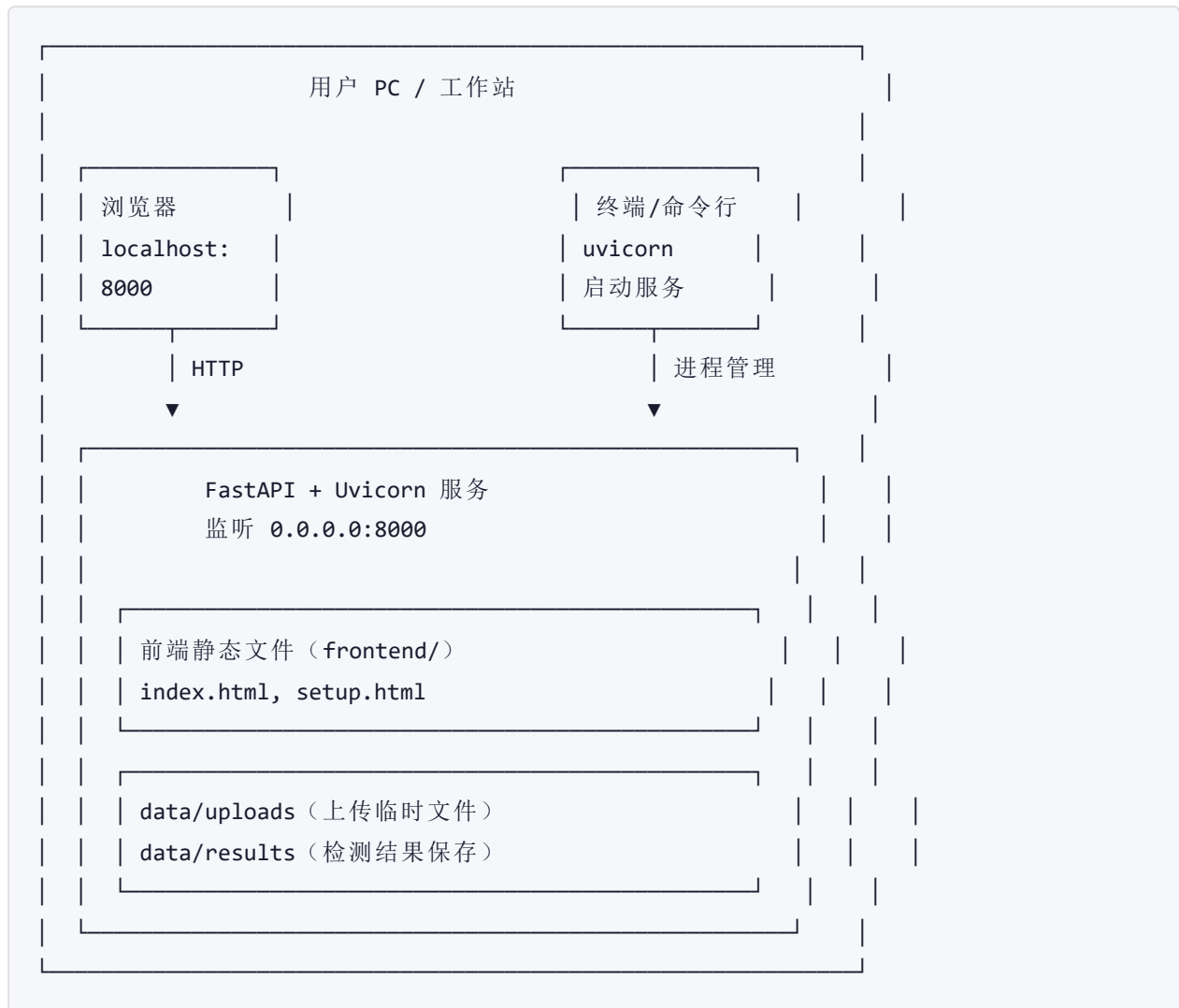
降级为像素特征 256维

(日志: [FaceRecognizer] 降级为像素特征匹配模式)

特征匹配关闭时的行为：当 `use_recognition=False` (默认)，Pipeline 不初始化 FaceRecognizer，所有人脸按检测顺序分配 person_id (1, 2, 3...)，无需降级。

7. 部署架构

7.1 本地开发部署



7.2 启动命令

```
cd face-detect-app/backend
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

启动后:

- 浏览器访问 `http://localhost:8000` 使用检测功能
- 手机配网页面: `http://localhost:8000/setup`

7.3 生产环境部署

生产环境采用 Nginx 反向代理共享 80 端口，Face Detect 服务挂载于 `/face/` 路径前缀。后端 FastAPI 服务监听 `127.0.0.1:8000`，不对外暴露。

服务器环境要求：

- OS: Linux (CentOS / Alibaba Cloud Linux / Ubuntu)
- Python: ≥ 3.10
- Nginx: ≥ 1.18
- 内存: $\geq 4\text{GB}$ (建议 7GB+, YOLO 模型加载约需 600MB)
- 磁盘: $\geq 5\text{GB}$ (含模型文件)

目录结构：

```
/opt/face-detect/
├── frontend/
│   ├── index.html
│   └── setup.html
├── backend/
│   ├── venv/
│   └── app/
│       ├── main.py
│       ├── detector.py
│       ├── recognizer.py
│       ├── seat_classifier.py
│       └── pipeline.py
├── data/
│   ├── uploads/
│   └── results/
├── yolov8n-face-lindevs.pt
└── requirements.txt

/usr/share/nginx/html/face-detect/    # Nginx 前端文件（软链接或复制）
/etc/nginx/conf.d/nozomi.conf         # Nginx 配置
/etc/systemd/system/face-detect.service # systemd 服务单元
```

systemd 服务配置：

```

[Unit]
Description=Face Detect API Service
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/face-detect/backend
ExecStart=/opt/face-detect/backend/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 2
Restart=always
RestartSec=5
Environment="PYTHONUNBUFFERED=1"
Environment="PUBLIC_IP=<公网IP地址>"
Environment="BASE_PATH=/face"

[Install]
WantedBy=multi-user.target

```

Nginx 代理配置:

```

location = /face/index.html {
    alias /usr/share/nginx/html/face-detect/index.html;
}
location = /face/setup.html {
    alias /usr/share/nginx/html/face-detect/setup.html;
}
location /face/ {
    alias /usr/share/nginx/html/face-detect/;
    index index.html;
    try_files $uri $uri/ =404;
}
location /face/api/ {
    proxy_pass http://127.0.0.1:8000/api/;
    client_max_body_size 50M;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

```

在线访问地址:

| 服务 | 地址 | |-----|-----| | Face Detect 主页面 | <http://<服务器IP>/face/> | | Face Detect 配
网页 | <http://<服务器IP>/face/setup> | | 原有服务 (Nozomi-House) | <http://<服务器IP>/> |

8. 设计约束与未来演进

8.1 当前设计约束

- **座位分类**：基于固定坐标比例规则，适用于摄像头安装位置和视角相对固定的车载场景。摄像头视角发生显著变化时需调整区域参数。
- **特征匹配默认关闭**：`use_recognition=False` 为默认值，不同请求之间不共享身份信息。如需跨图匹配需显式启用 InsightFace，CPU 模式下处理速度较慢。
- **单进程同步处理**：大批量上传时为同步阻塞模式，高并发场景存在性能瓶颈。
- **模型文件**：YOLOv8-face 模型文件（约 6MB）需随代码部署到服务器 `models/` 目录。
- **中文标注**：依赖系统字体文件，Linux 服务器需确保 `wqy-zenhei` 等中文字体已安装。

8.2 未来演进方向

近期可考虑：启用 InsightFace 特征匹配提升跨图身份精度；引入异步任务队列（Celery + Redis）提升批量并发能力；增加 COCO/VOC 格式导出功能。中期可增加：视频输入支持（自动抽帧处理）、WebSocket 实时进度推送、座位区域的前端可视化校准工具。远期可演进为多用户协作平台，支持标注结果的人工审核、修正和版本管理。